

Workshop 6: Observability Fundamentals

Wymagania wstępne

- 3-węzłowy klaster Kubernetes ze skonfigurowanym `kubectl` — wszystkie węzły muszą być w stanie `Ready`
 - **Zalecane środowisko:** klaster Google Cloud zbudowany przy użyciu skryptów z katalogu `scripts/gcp-scripts/` tego repozytorium; polecenia z `NODE_IP` w tych instrukcjach zakładają właśnie tę konfigurację
-

Przed rozpoczęciem

Upewnij się, że klaster działa i terminal ma skonfigurowany `kubectl` :

```
kubectl get nodes
```

Oczekiwany wynik — wszystkie trzy węzły w stanie `Ready` :

NAME	STATUS	ROLES	AGE	VERSION
k8s-workshop-master	Ready	control-plane	...	v1.30.0
k8s-workshop-worker-1	Ready	<none>	...	v1.30.0
k8s-workshop-worker-2	Ready	<none>	...	v1.30.0

Zapisz adres IP węzła roboczego — będzie potrzebny do otwierania interfejsów webowych w przeglądarce:

```
export NODE_IP=$(cd ~/prin-2026-observability/scripts/gcp-scripts/terraform &&  
terraform output -json worker_public_ips | python3 -c "import sys,json;  
print(json.load(sys.stdin)[0])")  
echo $NODE_IP
```

Dodaj go do profilu powłoki, żeby zmienna była dostępna po każdym ponownym połączeniu z Cloud Shell:

```
echo "export NODE_IP=$NODE_IP" >> ~/.bashrc
```

Ćwiczenie 1 — Wdrożenie Google Online Boutique

Online Boutique to aplikacja mikroserwisowa od Google składająca się z 10 usług. Symuluje sklep internetowy z usługami dla frontendu, koszyka, rekomendacji, płatności, wysyłki i innych. Każda usługa to osobny proces z własnym wdrożeniem, komunikujący się z pozostałymi przez sieć.

Kontekst telco: Ten wzorzec jest identyczny z tym, jak budowane są nowoczesne sieci 5G Core. Sieć 5G Core składa się z mikroserwisowych Network Functions — AMF (zarządzanie dostępem), SMF (zarządzanie sesjami), UPF (płaszczyzna użytkownika) i innych — z których każda działa jako niezależna usługa w Kubernetes, komunikując się z pozostałymi przez HTTP/gRPC. Implementacje open-source takie jak **free5GC** czy **Open5GS** stosują dokładnie ten wzorzec. Online Boutique jest prostsze do skonfigurowania na zajęciach, ale techniki obserwowalności, których się tu nauczysz, mają bezpośrednie zastosowanie w środowiskach 5G Core.

1.1 Utwórz namespace i wdróż aplikację

Czym jest namespace? Namespace Kubernetes to logiczna partycja klastra — sposób grupowania powiązanych zasobów i izolowania ich od innych grup. Pomyśl o tym jak o folderze. Możesz mieć namespace `boutique` dla aplikacji demo i namespace `monitoring` dla stosu obserwowalności — zasoby w jednym namespace nie widzą automatycznie zasobów w innym ani nie kolidują z nimi.

```
kubectl create namespace boutique
kubectl apply -n boutique -f https://raw.githubusercontent.com/GoogleCloudPlatform/microservices-demo/main/release/kubernetes-manifests.yaml
```

Sprawdź, czy zasoby są tworzone:

```
kubectl get pods -n boutique
kubectl get svc -n boutique
```

Na początku pody będą w stanie `ContainerCreating` lub `Pending` — to normalne. Kontynuuj czytanie i wróć za chwilę.

1.2 Udostępnij frontend przez NodePort

Domyślny manifest tworzy serwis typu `LoadBalancer` dla frontendu, który wymaga kontrolera load balancera w chmurze. Na klastrze kubeadm bez integracji z chmurą serwisy `LoadBalancer` nigdy nie otrzymują zewnętrznego IP (pozostają w stanie `Pending`).

NodePort to prostsze rozwiązanie: Kubernetes otwiera ten sam port na zewnętrznym IP każdego węzła. Ruch do `<dowolny-IP-węzła>:30080` jest przekierowywany do serwisu. To nie jest rozwiązanie produkcyjne — omija DNS, TLS i właściwy ingress — ale jest standardowym podejściem na zajęciach i lokalnych klastrach.

Alternatywa — użycie manifestu YAML zamiast `kubectl patch` :

Poniższe polecenie modyfikuje działający zasób za pomocą składni JSON Patch. Ten sam efekt można osiągnąć, edytując manifest YAML i stosując go przez `kubectl apply`. Obie metody są równoważne; `patch` jest szybszy przy jednorazowych zmianach, a manifest w pliku lepiej nadaje się do powtarzalnych, wersjonowanych wdrożeń. W prawdziwych projektach zazwyczaj zarządza się plikami YAML w repozytorium git.

```
kubectl patch svc frontend -n boutique \
--type='json' \
-p='[
  {"op":"replace","path":"/spec/type","value":"NodePort"},
  {"op":"add","path":"/spec/ports/0/nodePort","value":30080}
]'
```

Sprawdź, czy serwis został zmieniony:

```
kubectl get svc frontend -n boutique
```

Oczekiwany wynik: kolumna `TYPE` pokazuje `NodePort`, a `PORT(S)` pokazuje `80:30080/TCP`.

1.3 Poczekaj na uruchomienie podów

```
kubectl get pods -n boutique --watch
```

Wszystkie pody powinny osiągnąć stan `Running` w ciągu 3–5 minut. Naciśnij `Ctrl+C` gdy skończysz obserwować.

Jeśli pod pozostaje w stanie `CrashLoopBackOff` lub `ImagePullBackOff`, sprawdź szczegóły, aby zobaczyć przyczynę:

```
kubectl describe pod <nazwa-poda> -n boutique
```

1.4 Otwórz w przeglądarce

```
http://<NODE_IP>:30080
```

Powinieneś zobaczyć sklep Online Boutique. Przeglądaj — dodaj produkty do koszyka, przejdź do kasy — żeby wygenerować ruch HTTP między usługami. Stos monitoringu będzie miał co obserwować.

Czym jest Helm?

Przed wdrożeniem stosu monitoringu musisz poznać narzędzie, którego będziesz używać: **Helm**.

Helm to menedżer pakietów dla Kubernetes, podobny do `apt` na Ubuntu czy `pip` w Pythonie. Zamiast pisać i utrzymywać dziesiątki pojedynczych manifestów YAML, instalujesz **chart** — gotową paczkę manifestów z konfigurowalnymi wartościami domyślnymi.

Kluczowe pojęcia:

Pojęcie	Znaczenie
Chart	Pakiet manifestów Kubernetes (jak <code>.deb</code> lub <code>.whl</code>)
Repository	Serwer przechowujący charty (jak PyPI lub repozytoria apt)
Release	Wdrożona instancja charta w klastrze
Values	Parametry konfiguracyjne — nadpisujesz wartości domyślne przez <code>--set klucz=wartość</code> lub plik <code>values.yaml</code>

Najczęściej używane flagi:

Flaga	Działanie
<code>--namespace <ns></code>	Wdróż zasoby do tego namespace
<code>--create-namespace</code>	Utwórz namespace, jeśli nie istnieje
<code>--set key=value</code>	Nadpisz pojedynczą wartość charta
<code>--values file.yaml</code>	Nadpisz wiele wartości z pliku
<code>--wait</code>	Czekaj, aż wszystkie pody będą Running przed zakończeniem
<code>--timeout 5m</code>	Anuluj instalację po upływie tego czasu, jeśli klaster nie osiągnie gotowości

Aby wyświetlić zainstalowane release w danym namespace: `helm list -n <namespace>` .
Aby usunąć release: `helm uninstall <nazwa-release> -n <namespace>` .

Oficjalna dokumentacja: helm.sh/docs — Chart Template Guide to najlepsze miejsce na start, gdy chcesz pisać własne charty.

Chart to katalog o stałej strukturze:

```
mychart/
├─ Chart.yaml           # nazwa, wersja, opis
├─ values.yaml          # domyślne wartości konfiguracyjne
├─ templates/           # manifesty Kubernetes ze składnią szablonów Go
│   └─ deployment.yaml
│   └─ service.yaml
│   └─ _helpers.tpl     # współdzielone helpery szablonów
└─ charts/              # opcjonalne zależności (sub-charty)
```

Podczas `helm install` Helm przetwarza szablony — w miejsce referencji `{{ .Values.* }}` wstawia odpowiednie wartości — i wysyła gotowe manifesty do Kubernetes API, które tworzy opisane w nich zasoby.

Ćwiczenie 2 — Wdrożenie stosu monitoringu

Wdrożysz sześć komponentów do namespace `monitoring`:

Komponent	Rola
Prometheus	Zbiera i przechowuje metryki
Grafana	Wizualizuje metryki, logi i ślady — ujednolicony interfejs
Alertmanager	Kieruje alerty (dołączony do kube-prometheus-stack)
Loki	Przechowuje i indeksuje logi
Promtail	Czyta logi podów z dysku i wysyła je do Loki
Tempo	Przechowuje ślady rozproszone

2.1 Dodaj repozytoria Helm

Repozytorium to serwer HTTPS z listą dostępnych chartów. Dodajesz je raz — Helm buforuje indeks lokalnie.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Sprawdź, czy repozytoria zostały dodane:

```
helm repo list
```

2.2 Wdróż kube-prometheus-stack

kube-prometheus-stack to jeden chart Helm, który wdraża od razu pięć komponentów: Prometheus, Grafana, Alertmanager, **node-exporter** (zbiera metryki sprzętowe z każdego węzła) i **kube-state-metrics** (eksponuje stan obiektów Kubernetes — status Deploymentów, liczba podów itp. — jako metryki Prometheusa).

Instalowanie ich osobno wymagałoby koordynacji pięciu niezależnych release i ręcznego łączenia konfiguracji. Ten chart robi to za Ciebie i od razu dostarcza dziesiątki gotowych dashboardów.

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --set grafana.adminPassword=workshop123 \
  --set grafana.service.type=NodePort \
  --set grafana.service.nodePort=30300 \
  --set prometheus.prometheusSpec.retention=6h \
  --wait --timeout 5m
```

`grafana.service.type=NodePort` i `grafana.service.nodePort=30300` — udostępniają Grafanę na porcie 30300 każdego węzła, żebyś mógł ją otworzyć w przeglądarce (ta sama zasada co NodePort dla Online Boutique).

`prometheus.prometheusSpec.retention=6h` — przechowuj tylko 6 godzin danych metryk; to klaster warsztatowy, nie produkcja.

Sprawdź release i jego pody:

```
helm list -n monitoring
kubectl get pods -n monitoring
```

Oczekiwany wynik: jeden release `kube-prometheus-stack-*` i kilka podów — Prometheus, Grafana, Alertmanager, node-exporter (jeden na węzeł), kube-state-metrics — wszystkie w stanie `Running`.

2.3 Wdróż Loki

Loki to backend przechowywania logów Grafany. Celowo **nie** indeksuje pełnej treści logów (w przeciwieństwie do Elasticsearch) — indeksuje tylko etykiety i skanuje treść logów dopiero podczas zapytania. Takie podejście spowalnia zapytania, ale drastycznie obniża koszty przechowywania. W Kubernetes, gdzie liczba różnych wartości etykiet jest ograniczona, to rozsądny kompromis.

Dlaczego Loki to osobny chart? Bo to osobny produkt z własnym cyklem wydań, wersjonowaniem i schematem konfiguracji. Stos Grafany to nie monolit — to zbiór niezależnych komponentów komunikujących się przez otwarte protokoły. Połączenie między nimi skonfigurujesz ręcznie w Ćwiczeniu 3.

--set VS --values : Instalacja kube-prometheus-stack powyżej użyła `--set klucz=wartość` dla kilku prostych nadpisań — wygodne przy krótkich jednolinijkowcach. Loki ma więcej ustawień, więc zapisujemy je do pliku `values.yaml` i przekazujemy przez `--values`. Oba podejścia nadpisują wartości domyślne charta; `--values` jest wygodniejszy, gdy opcji jest więcej niż dwie lub trzy, a plik można trzymać w repozytorium git.

Przejrzyj plik `values` przed instalacją — upewnij się, że rozumiesz, co robi każdy klucz:

```
cat ~/prin-2026-observability/manifests/loki-values.yaml
```

`deploymentMode: SingleBinary` — uruchamia wszystkie komponenty Loki w jednym podzie. Loki obsługuje trzy tryby: **SingleBinary** (jeden pod, wszystko razem — dla warsztatów i testów), **SimpleScalable** (osobne pody read/write/backend) i **Distributed** (w pełni mikroserwisowy, dla bardzo dużej skali). Domyślne liczby replik dla `read`, `write` i `backend` wynoszą 1 w chartcie, co kolidowałoby z trybem `SingleBinary` — ustawiamy je jawnie na 0.

`useTestSchema: true` — Loki wymaga schematu przechowywania określającego sposób indeksowania fragmentów logów w czasie. Na produkcji definiujesz go samodzielnie; na warsztatach wbudowany schemat testowy wystarczy.

`persistence.enabled: false` — brak wolumenu trwałego; logi są przechowywane w pamięci. Giną po restarcie poda — akceptowalne na warsztatach.

Instalacja:

```
helm install loki grafana/loki \
  --namespace monitoring \
  --values ~/prin-2026-observability/manifests/loki-values.yaml \
  --wait --timeout 5m
```

Sprawdź:

```
kubectl get pods -n monitoring -l app.kubernetes.io/name=loki
```

Oczekiwany wynik: jeden pod `loki-0` w stanie `Running`. Zwróć uwagę, że nazwa kończy się na `-0` — ta liczba to indeks poda w **StatefulSet**.

Deployment vs StatefulSet: Kubernetes ma dwa główne typy workloadów do uruchamiania podów:

	Deployment	StatefulSet
Nazwy podów	Losowy sufiks (<code>frontend-7d9f8b-xkp2</code>)	Stabilny indeks (<code>loki-0</code> , <code>loki-1</code>)
Przechowywanie	Współdzielone lub brak	Każdy pod ma własny PersistentVolumeClaim
Uruchamianie/ wyłączanie	Równoległe, dowolna kolejność	Sekwencyjne, uporządkowane
Typowe zastosowanie	Usługi bezstanowe (frontend, API)	Usługi stanowe (bazy danych, przechowywanie)

Loki jest wdrożony jako StatefulSet, bo zarządza własnym przechowywaniem na dysku i każda replika potrzebuje stabilnej tożsamości. Prometheus też jest StatefulSet z tego samego powodu. Promtail (który wdrożysz dalej) to DaemonSet — trzeci typ, który zapewnia dokładnie jeden pod na każdym węźle.

2.4 Wdróż Promtail

Promtail to agent zbierający logi. Działa jako **DaemonSet** — Kubernetes wdraża dokładnie jeden pod Promtail na każdym węźle, więc żadne logi żadnego poda nie zostaną pominięte, niezależnie od tego, na którym węźle wylądował.

Promtail czyta pliki logów z `/var/log/pods/` na systemie plików hosta (te same pliki, które czytasz przez `kubectl logs`), dołącza metadane Kubernetes jako etykiety (namespace, nazwa poda, kontener) i wysyła strumień logów do Loki.

```
helm install promtail grafana/promtail \
  --namespace monitoring \
  --set config.clients[0].url=http://loki:3100/loki/api/v1/push \
  --wait --timeout 3m
```

Sprawdź:

```
kubectl get pods -n monitoring -l app.kubernetes.io/name=promtail -o wide
```

Oczekiwany wynik: jeden pod Promtail na węzeł (trzy łącznie). Kolumna `NODE` potwierdza, że każdy pod wylądował na innym węźle — tak działa DaemonSet.

2.5 Wdróż Tempo

Tempo przechowuje ślady rozproszone. Nie ma jeszcze danych — instrumentacja OTel zostanie skonfigurowana później. Wdrożenie Tempo teraz oznacza, że Grafana może być od razu skonfigurowana z Tempo jako źródłem danych.

```
helm install tempo grafana/tempo \
  --namespace monitoring \
  --set tempo.storage.trace.backend=local \
  --wait --timeout 3m
```

Sprawdź:

```
kubectl get pods -n monitoring -l app.kubernetes.io/name=tempo
```

Oczekiwany wynik: jeden pod `tempo-0` w stanie `Running`.

2.6 Finalna weryfikacja

Sprawdź wszystkie komponenty monitoringu naraz:

```
kubectl get pods -n monitoring
```

Wszystkie pody powinny być w stanie `Running`. Jeśli pod jest w stanie `Pending`, sprawdź zasoby lub przeciążenie węzła:

```
kubectl describe pod <nazwa-poda> -n monitoring
```

Sprawdź, czy Grafana jest dostępna — odwiedź `http://<NODE_IP>:30300` w przeglądarce. Powinieneś zobaczyć ekran logowania Grafany.

Ćwiczenie 3 — Konfiguracja źródeł danych Grafany

Otwórz Grafanę:

```
http://<NODE_IP>:30300
```

Login: **admin** / **workshop123**

kube-prometheus-stack skonfigurował już Prometheusa jako źródło danych. Musisz ręcznie dodać Loki i Tempo — tu właśnie łączysz niezależne komponenty.

DNS wewnątrz klastra: Adresy URL poniżej używają wewnętrznych nazw DNS Kubernetes, takich jak `loki.monitoring.svc.cluster.local`. W Kubernetes każdy Serwis otrzymuje nazwę DNS w formacie `<nazwa-serwisu>.<namespace>.svc.cluster.local`. Pody wewnątrz klastra używają tej nazwy do komunikacji ze sobą, niezależnie od tego, na którym węźle są zaplanowane. Krótka forma `http://loki:3100` też działa, gdy wywołujący jest w tym samym namespace, ale pełna forma jest jednoznaczna i zawsze działa.

3.1 Dodaj Loki

1. Przejdź do **Connections** → **Data sources** → **Add new data source**
2. Wybierz **Loki**
3. Ustaw URL: `http://loki.monitoring.svc.cluster.local:3100`
4. Kliknij **Save & test** — powinieneś zobaczyć "Data source successfully connected"

3.2 Dodaj Tempo

1. **Add new data source** → **Tempo**
2. Ustaw URL: `http://tempo.monitoring.svc.cluster.local:3200`
3. Kliknij **Save & test**

Ćwiczenie 4 — Eksploracja gotowych dashboardów

kube-prometheus-stack dostarcza dziesiątki gotowych dashboardów — to jeden z kluczowych powodów używania zbiorczego charta zamiast ręcznego wdrażania Prometheusa.

Przejdź do **Dashboards** na lewym pasku bocznym. Otwórz każdy z poniższych i poświęć kilka minut na zrozumienie, jakie metryki są pokazane i dlaczego:

1. **Kubernetes / Nodes** — CPU, pamięć, I/O dysku na węzeł
2. **Kubernetes / Pods** — użycie zasobów na pod
3. **Kubernetes / Compute Resources / Namespace (Pods)** — wybierz namespace `boutique`, żeby zobaczyć wszystkie usługi Online Boutique

Te dashboardy są zasilane przez **kube-state-metrics** (stan obiektów: ile podów działa, ile jest oczekiwanych, ile gotowych) i **node-exporter** (metryki sprzętowe: rzeczywiste sekundy CPU, bajty pamięci). Żaden z nich nie jest Twoją aplikacją — to osobne eksportery udostępniające wewnętrzne informacje Kubernetes i systemu operacyjnego w formacie Prometheusa.

Spróbuj zmienić zakres czasu (górny prawy róg) na **Last 15 minutes** i włącz **Auto refresh every 10s**.

4.1 Eksploracja logów w Grafanie

Przejdź do **Explore** (ikona kompasu na pasku bocznym).

1. Wybierz źródło danych **Loki** z listy rozwijanej
2. W przeglądarce etykiet wybierz `namespace = boutique`
3. Kliknij **Run query** — powinieneś zobaczyć linie logów ze wszystkich podów Online Boutique
4. Filtruj do jednej usługi, zmieniając zapytanie na `{namespace="boutique", app="frontend"}` i kliknij **Run query** ponownie
5. Szukaj linii z błędami: dodaj `|= "error"` na końcu zapytania (wielkość liter ma znaczenie) i uruchom ponownie

To jest zapytanie **LogQL** — język zapytań Loki. Część `{...}` wybiera strumienie według etykiet; `|=` filtruje te strumienie według zawartości tekstu. Jeśli znasz PromQL Prometheusa, LogQL stosuje ten sam wzorec selektorów etykiet, ale dodaje operacje na strumieniach logów.

Ćwiczenie 5 — Budowanie własnego dashboardu

Zbudujesz prosty dashboard pokazujący użycie zasobów przez Online Boutique. Celem jest zrozumienie, jak działają zapytania PromQL i jak budowane są panele Grafany — umiejętności przydatne w każdym prawdziwym projekcie.

5.1 Utwórz nowy dashboard

Przejdź do **Dashboards** → **New** → **New dashboard**. Zobaczysz pusty dashboard — kliknij w wyznaczone miejsce na środku ekranu, żeby dodać pierwszy panel.

Wybierz źródło danych **Prometheus**.

5.2 Panel 1 — Użycie CPU na usługę

Przełącz się do trybu **Code** (górny prawy róg edytora zapytań) i wpisz:

```
sum(rate(container_cpu_usage_seconds_total{namespace="boutique", container!=""}[2m]))
by (pod)
```

Czytanie tego zapytania:

- `container_cpu_usage_seconds_total` — licznik, który stale rośnie wraz z każdą sekundą CPU zużytą przez kontener
- `rate(...[2m])` — oblicza tempo wzrostu licznika w ciągu ostatnich 2 minut i zwraca wynik w jednostkach na sekundę
- `sum(...) by (pod)` — sumuje wyniki dla wszystkich kontenerów w ramach poda, dając jedną linię na usługę

Ustaw **Title**: `Użycie CPU – Online Boutique`. Ustaw wizualizację na **Time series**. Kliknij **Apply**.

5.3 Panel 2 — Użycie pamięci na usługę

Dodaj drugi panel:

```
sum(container_memory_working_set_bytes{namespace="boutique", container!=""}) by (pod)
```

`working_set_bytes` to faktycznie używana pamięć (bez cache). W przeciwieństwie do CPU, pamięć to wskaźnik — może rosnąć i maleć — więc `rate()` nie jest potrzebny.

Ustaw **Title**: `Użycie pamięci – Online Boutique`. W **Standard options** ustaw **Unit** na `bytes(IEC)` — Grafana automatycznie sformatuje jako MB/GB.

Kliknij **Apply**, a następnie **Save dashboard** (nazwij go `Online Boutique Overview`).

5.4 Panel 3 — Restarty podów

Dodaj trzeci panel:

```
round(increase(kube_pod_container_status_restarts_total{namespace="boutique"}[10m]))
```

`increase()` działa jak `rate()`, ale zwraca łączny wzrost w oknie czasowym zamiast szybkości na sekundę. Wartość `2` oznacza, że pod restartował się dwukrotnie w ciągu ostatnich 10 minut.

Ustaw **Title**: `Restarty podów (ostatnie 10 min)`. Ustaw wizualizację na **Bar gauge** — jeden pasek na pod, co natychmiast pokazuje który pod restartuje się najczęściej. Opcjonalnie ustaw progi: 0 = zielony, 1-3 = żółty, >3 = czerwony.

Zapisz dashboard. Ten panel będzie przydatny w następnym ćwiczeniu, gdy zaczniesz celowo wprowadzać awarie.

Ćwiczenie 6 — Wdrożenie Chaos Mesh i wstrzyknięcie awarii

Chaos Mesh to narzędzie do **inżynierii chaosu**. Idea polega na celowym wprowadzaniu awarii w wybranych komponentach systemu, a następnie weryfikacji, czy monitoring je wykrywa i czy system poprawnie wraca do normalnego stanu. W produkcji ta praktyka potwierdza, że runbooki, alerty i narzędzia obserwowalności naprawdę działają — zanim dojdzie do prawdziwego incydentu.

Zasady inżynierii chaosu opisane są w manifeście **Principles of Chaos Engineering**: principlesofchaos.org. Kluczowa idea to budowanie pewności co do zachowania systemu poprzez eksperymenty — zaczynając od hipotezy ("system pozostanie stabilny, gdy usługa X padnie"), przeprowadzając kontrolowany eksperyment i porównując rzeczywiste zachowanie z hipotezą.

6.1 Wdróż Chaos Mesh

```
helm repo add chaos-mesh https://charts.chaos-mesh.org
helm repo update

helm install chaos-mesh chaos-mesh/chaos-mesh \
  --namespace chaos-mesh \
  --create-namespace \
  --set chaosDaemon.runtime=containerd \
  --set chaosDaemon.socketPath=/run/containerd/containerd.sock \
  --set dashboard.service.type=NodePort \
  --set dashboard.service.nodePort=32688 \
  --wait --timeout 5m
```

`chaosDaemon.runtime=containerd` i `socketPath` — demon Chaos Mesh musi komunikować się bezpośrednio ze środowiskiem uruchomieniowym kontenerów, żeby je zatrzymywać. Flagi powyżej są poprawne dla klastra zbudowanego skryptami warsztatowymi, który używa **containerd**.

Jeśli używasz własnego klastra z innym środowiskiem uruchomieniowym, sprawdź najpierw, co jest zainstalowane:

```
kubectl get nodes -o wide
```

Kolumna `CONTAINER-RUNTIME` pokazuje środowisko i wersję (np. `containerd://1.7.x`, `docker://24.x`, `cri-o://1.x`). Użyj odpowiednich wartości:

Środowisko	--set chaosDaemon.runtime	--set chaosDaemon.socketPath
containerd	containerd	/run/containerd/containerd.sock
Docker	docker	/var/run/docker.sock
CRI-O	crio	/var/run/crio/crio.sock

Sprawdź:

```
kubectl get pods -n chaos-mesh
```

Oczekiwany wynik: pody `chaos-controller-manager`, `chaos-daemon` (jeden na węzeł) i `chaos-dashboard` w stanie `Running`.

6.2 Otwórz dashboard Chaos Mesh i zaloguj się

```
http://<NODE_IP>:32688
```

Dashboard wymaga tokenu Kubernetes do logowania. To jest **RBAC (Role-Based Access Control)** — system uprawnień Kubernetes. Każda akcja w klastrze jest wykonywana przez **ServiceAccount** ze zdefiniowanym zestawem uprawnień (**Role** lub **ClusterRole**). Dashboard wymaga tokenu potwierdzającego, że masz uprawnienia do tworzenia eksperymentów chaosu.

Kliknij **Click here to generate** na ekranie logowania. Ustaw:

- **Cluster scoped**: zaznaczone (pozwala zarządzać eksperymentami we wszystkich namespace)
- **Role**: `manager` (pozwala tworzyć i usuwać eksperymenty, nie tylko przeglądać)

Dashboard pokaże manifest RBAC i polecenia potrzebne do utworzenia ServiceAccount i pobrania jego tokenu. Uruchom te polecenia w terminalu, wklej token do pola logowania i kliknij **Submit**.

Zapoznaj się z interfejsem — możesz tu tworzyć eksperymenty. W tym ćwiczeniu użyjesz też manifestu YAML bezpośrednio, co jest bardziej powtarzalne niż UI.

6.3 Custom Resource Definitions

Do tej pory pracowałeś z wbudowanymi typami obiektów Kubernetes: Deployments, DaemonSets, StatefulSets, Services. Ale Kubernetes jest rozszerzalny — każdy może definiować nowe typy obiektów i rejestrować je w API serverze. Nazywają się **Custom Resource Definitions (CRDs)**.

CRD to deklaracja schematu mówiąca Kubernetesowi: "istnieje nowy rodzaj obiektu o nazwie X i ma te pola." Po zarejestrowaniu możesz tworzyć, listować i usuwać obiekty tego typu przez `kubectl` tak samo jak wbudowane zasoby. Operatory używają CRDs, żeby pozwolić opisywać pożądany stan w YAML, a następnie na nim działać.

Wylistuj wszystkie CRDs zarejestrowane w klastrze:

```
kubectl get crd
```

Filtruj do CRDs Chaos Mesh:

```
kubectl get crd | grep chaos-mesh
```

Powinieneś zobaczyć kilka typów: `podchaos`, `networkchaos`, `httpchaos`, `stresschaos` i inne — każdy reprezentuje inną kategorię eksperymentu.

Przejrzyj schemat typu `PodChaos`, żeby zobaczyć, jakie pola akceptuje:

```
kubectl explain podchaos.spec
```

Możesz zagłębić się dalej, na przykład:

```
kubectl explain podchaos.spec.mode
```

`kubectl explain` działa dla każdego zasobu — wbudowanego lub własnego — i jest najszybszym sposobem sprawdzenia dostępnych pól bez wychodzenia z terminala.

6.4 Uruchom eksperyment pod-failure

Poniższy manifest definiuje eksperyment `PodChaos`, który zabija pod `recommendationservice` co 60 sekund i działa do momentu ręcznego usunięcia Schedule.

YAML vs UI: Dashboard Chaos Mesh i podejście z `kubectl apply` dają identyczne rezultaty — oba tworzą obiekt `PodChaos` w Kubernetes. Podejście z manifestem jest powtarzalne i można je commitować do gita. UI jest przydatne do eksploracji.

W Chaos Mesh v2 powtarzające się eksperymenty definiuje się przez zasób `Schedule` — CRD, który opakowuje dowolny typ eksperymentu chaosu i dodaje możliwość cyklicznego uruchamiania (podobnie jak cron). Blok `podChaos` wewnątrz ma identyczną strukturę co samodzielny zasób `PodChaos`.

Poniższy eksperyment używa `pod-failure`, który powoduje awarię kontenerów wewnątrz poda i ich restart przez kubelet — obiekt poda pozostaje, problemy dotyczą tylko jego kontenerów. To różni się od `pod-kill`, który całkowicie usuwa pod i każe Kubernetesowi stworzyć nowy.

```
cat ~/prin-2026-observability/manifests/crash-recommendation.yaml

kubectl apply -f ~/prin-2026-observability/manifests/crash-recommendation.yaml
```

Sprawdź, czy eksperyment został utworzony:

```
kubectl get schedule -n boutique
```

6.5 Obserwuj wpływ w Grafanie

Wróć do swojego dashboardu **Online Boutique Overview**. Po minucie lub dwóch powinieneś zobaczyć:

- Panel **Restarty podów** ze skokiem, gdy kontenery ulegają awarii i są restartowane przez kubelet
- Panel **Użycie pamięci** z krótkim spadkiem podczas awarii i odbudową po powrocie kontenerów

Otwórz też widok **Explore** → Loki → zapytanie `{namespace="boutique", app="recommendationservice"}` i obserwuj, jak logi zatrzymują się i wznowiają z każdym cyklem awarii.

6.6 Posprzątaj po eksperymencie

```
kubectl delete schedule crash-recommendation -n boutique
```

Sprawdź, czy pod się stabilizuje:

```
kubectl get pods -n boutique -l app=recommendationservice
```

Ćwiczenie 7 — Challenge: Dashboard Dyżurnego

Zbuduj dashboard Grafany, który pozwoli dyżurnemu inżynierowi odpowiedzieć na pytanie "czy Online Boutique działa prawidłowo w tej chwili?" w ciągu 30 sekund od otwarcia — bez wcześniejszej znajomości systemu.

Elementy, które warto rozważyć:

- **Stan podów** — dla każdej usługi boutique: czy pody działają i ile restartów nastąpiło niedawno. Skok tutaj to pierwszy sygnał, że coś się dzieje.
- **Obciążenie zasobów** — zużycie CPU i pamięci na usługę. Mówi to, czy problem jest spowodowany skokiem obciążenia, czy powolnym wyciekiem pamięci.
- **Sygnał błędów z logów** — widok ostatnich linii logów na poziomie error z namespace boutique. Metryki mówią że coś jest nie tak; logi mówią co.
- **Jeden wskaźnik kondycji na pierwszy rzut oka** — jeden panel, który zmęczony inżynier może sprawdzić jako pierwszy. Pomyśl, jaka pojedyncza liczba lub kolor najlepiej podsumowuje stan całej aplikacji.

Jak wygląda dobry wynik:

Wszystkie panele mają znaczące tytuły, poprawne jednostki (bajty, nie surowe liczby; wartości procentowe tam, gdzie to właściwe) i zakres czasu odpowiedni dla reagowania na incydenty — nie dni, nie sekundy.

Gdy wszystko działa, dashboard jest spokojny i zielony. Gdy usługa przestaje działać poprawnie, odpowiednie panele zmieniają się widocznie — bez konieczności wskazywania przez Ciebie konkretnego miejsca.

Weryfikacja: gdy skończysz, wróć do Chaos Mesh i wprowadź awarię na dowolną usługę. Możesz użyć eksperymentu `pod-failure` z Ćwiczenia 6 lub wybrać inny typ — `pod-kill`, `network-delay` czy `stress`. Dashboard powinien sam wskazać, która usługa jest dotknięta i kiedy wróciła do normalnego działania.